

Import Packages

In [1]: `%pip install PyWavelets`

Requirement already satisfied: PyWavelets in c:\users\rd908\onedrive - purdue.edu\graduate school\spring 2026\me-597-iiot\.venv\lib\site-packages (1.9.0)

Requirement already satisfied: numpy<3,>=1.25 in c:\users\rd908\onedrive - purdue.edu\graduate school\spring 2026\me-597-iiot\.venv\lib\site-packages (from PyWavelets) (2.4.1)

Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 25.2 -> 26.0.1

[notice] To update, run: python.exe -m pip install --upgrade pip

In [2]: `import pandas as pd
import numpy as np
import scipy.stats as sp
import pywt`

Declare the size of feature dataset

In [3]: `NoOfData = (
 180 # 180 Data for each robotic spot-welding condition (Normal, Abnormal)
)
NoOfSensor = 3 # 3 Sensor signals: Acceleration, Voltage, Current
NoOfFeature = 10 # 10 Feature types: Max, Min, Mean, RMS, Variance, Skewness, Kurtosis, Crest factor, Shape

NoOfData, NoOfSensor, NoOfFeature`

Out[3]: (180, 3, 10)

Load Raw Dataset (360 files)

In [4]: `from urllib.error import HTTPError

for i in range(NoOfData):

 temp_path1 = f"https://github.com/purduelamm/purdue_me597_iiot/blob/main/ml_tutorial/Dataset/Normal_{i+1}"
 temp_path2 = f"https://github.com/purduelamm/purdue_me597_iiot/blob/main/ml_tutorial/Dataset/Abnormal_{i+1}"

 try:
 exec(
 f"Normal_{i+1} = pd.read_csv(temp_path1 , sep=',', header=None)"
)
 except HTTPError:
 print(f"Failed to load data for file Normal_{i+1}")
 try:
 exec(
 f"Abnormal_{i+1} = pd.read_csv(temp_path2 , sep=',', header=None)"
)
 except HTTPError:
 print(f"Failed to load data for file Abnormal_{i+1}")`

Time Domain Feature Extraction

- 10 features * 3 sensors = 30 features

In [5]: `# Definition of rms function
def rms(x):
 return np.sqrt(np.mean(x**2))`

In [6]: `# Create empty(0) arrays for normal/abnormal feature dataset (time domain)
TimeFeature_Normal = np.zeros((NoOfSensor * NoOfFeature, NoOfData))`

```
TimeFeature_Abnormal = np.zeros((NoOfSensor * NoOfFeature, NoOfData))

print(TimeFeature_Normal.shape)
print(TimeFeature_Abnormal.shape)

TimeFeature_Normal
```

```
(30, 180)
(30, 180)
```

```
Out[6]: array([[0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               ...,
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.]], shape=(30, 180))
```

```
In [7]: for i in range(NoOfData):

        # Declare temporary data
        exec(f"temp_data1 = Normal_{i+1}")
        exec(f"temp_data2 = Abnormal_{i+1}")

        # Time domain feature extraction
        for j in range(NoOfSensor):

            # Normal features
            TimeFeature_Normal[NoOfFeature * j + 0, i] = np.max(
                temp_data1.iloc[:, j + 1]
            )
            TimeFeature_Normal[NoOfFeature * j + 1, i] = np.min(
                temp_data1.iloc[:, j + 1]
            )
            TimeFeature_Normal[NoOfFeature * j + 2, i] = np.mean(
                temp_data1.iloc[:, j + 1]
            )
            TimeFeature_Normal[NoOfFeature * j + 3, i] = rms(
                temp_data1.iloc[:, j + 1]
            )
            TimeFeature_Normal[NoOfFeature * j + 4, i] = np.var(
                temp_data1.iloc[:, j + 1]
            )
            TimeFeature_Normal[NoOfFeature * j + 5, i] = sp.skew(
                temp_data1.iloc[:, j + 1]
            )
            TimeFeature_Normal[NoOfFeature * j + 6, i] = sp.kurtosis(
                temp_data1.iloc[:, j + 1]
            )
            TimeFeature_Normal[NoOfFeature * j + 7, i] = np.max(
                temp_data1.iloc[:, j + 1]
            ) / rms(temp_data1.iloc[:, j + 1])
            TimeFeature_Normal[NoOfFeature * j + 8, i] = rms(
                temp_data1.iloc[:, j + 1]
            ) / np.mean(np.abs(temp_data1.iloc[:, j + 1]))
            TimeFeature_Normal[NoOfFeature * j + 9, i] = np.max(
                temp_data1.iloc[:, j + 1]
            ) / np.mean(np.abs(temp_data1.iloc[:, j + 1]))

            # Abnormal features
            TimeFeature_Abnormal[NoOfFeature * j + 0, i] = np.max(
                temp_data2.iloc[:, j + 1]
            )
            TimeFeature_Abnormal[NoOfFeature * j + 1, i] = np.min(
                temp_data2.iloc[:, j + 1]
            )
            TimeFeature_Abnormal[NoOfFeature * j + 2, i] = np.mean(
                temp_data2.iloc[:, j + 1]
            )
            TimeFeature_Abnormal[NoOfFeature * j + 3, i] = rms(
                temp_data2.iloc[:, j + 1]
            )
```

```

    )
    TimeFeature_Abnormal[NoOfFeature * j + 4, i] = np.var(
        temp_data2.iloc[:, j + 1]
    )
    TimeFeature_Abnormal[NoOfFeature * j + 5, i] = sp.skew(
        temp_data2.iloc[:, j + 1]
    )
    TimeFeature_Abnormal[NoOfFeature * j + 6, i] = sp.kurtosis(
        temp_data2.iloc[:, j + 1]
    )
    TimeFeature_Abnormal[NoOfFeature * j + 7, i] = np.max(
        temp_data2.iloc[:, j + 1]
    ) / rms(temp_data2.iloc[:, j + 1])
    TimeFeature_Abnormal[NoOfFeature * j + 8, i] = rms(
        temp_data2.iloc[:, j + 1]
    ) / np.mean(np.abs(temp_data2.iloc[:, j + 1]))
    TimeFeature_Abnormal[NoOfFeature * j + 9, i] = np.max(
        temp_data2.iloc[:, j + 1]
    ) / np.mean(np.abs(temp_data2.iloc[:, j + 1]))

print(TimeFeature_Normal.shape)
print(TimeFeature_Abnormal.shape)

TimeFeature_Normal

```

```
(30, 180)
```

```
(30, 180)
```

```

Out[7]: array([[ 1.35100000e+00,  3.16610000e+01,  3.18320000e+01, ...,
                1.05340000e+00,  1.08740000e+00,  9.58390000e-01],
               [-1.37200000e+00, -2.27860000e+01, -2.36130000e+01, ...,
               -9.65520000e-01, -1.16450000e+00, -8.99710000e-01],
               [ 1.10828100e-02,  2.33385723e-02,  2.05055037e-02, ...,
                3.09079384e-02,  3.20263227e-02,  3.37205056e-02],
               ...,
               [ 1.90403829e+00,  1.93966077e+00,  1.94395765e+00, ...,
                1.89501956e+00,  1.91625999e+00,  1.90413317e+00],
               [ 1.44617765e+00,  1.44713933e+00,  1.44831636e+00, ...,
                1.44636143e+00,  1.44619809e+00,  1.44548575e+00],
               [ 2.75357761e+00,  2.80695940e+00,  2.81546567e+00, ...,
                2.74088319e+00,  2.77129154e+00,  2.75239737e+00]],
          shape=(30, 180))

```

Combine Normal and Abnormal feature arrays

- axis=0: combine rows
- axis=1: combine columns

```

In [8]: TimeFeature = np.concatenate(
        [TimeFeature_Normal, TimeFeature_Abnormal], axis=1
    )
TimeFeature.shape

```

```
Out[8]: (30, 360)
```

Frequency Domain Feature Extraction

- 10 features _ 8 wavelet levels _ 3 sensors = 240 features

```

In [9]: # Wavelet options
MotherWavelet = pywt.Wavelet("haar") # Mother wavelet
Level = 8 # Wavelet decomposition Level

```

```

In [10]: # Create empty(0) arrays for normal/abnormal feature dataset (frequency Domain)
FreqFeature_Normal = np.zeros(
    shape=(NoOfSensor * NoOfFeature * Level, NoOfData)
)

```

```

FreqFeature_Abnormal = np.zeros(
    shape=(NoOfSensor * NoOfFeature * Level, NoOfData)
)

print(FreqFeature_Normal.shape)
print(FreqFeature_Abnormal.shape)

FreqFeature_Normal

```

```
(240, 180)
```

```
(240, 180)
```

```

Out[10]: array([[0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               ...,
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.]], shape=(240, 180))

```

```

In [11]: for i in range(NoOfData):

    # Declare temporary data (only sensor signals)
    exec(f"temp_data1 = Normal_{i+1}.iloc[:,1:]")
    exec(f"temp_data2 = Abnormal_{i+1}.iloc[:,1:]")

    # Walvelet decomposition
    Coef1 = pywt.wavedec(temp_data1, MotherWavelet, level=Level, axis=0)
    Coef2 = pywt.wavedec(temp_data2, MotherWavelet, level=Level, axis=0)

    # Frequency domain feature extraction
    for j in range(NoOfSensor):

        for k in np.arange(Level):
            coef1 = Coef1[Level - k]
            coef2 = Coef2[Level - k]

            #####
            # Complete code below to obtain proper features
            # Tip: Use NoOfFeature, Level, j, and k
            #####
            # Assuming the indexes of max, min, mean, etc. are
            # the same as time domain features, but with an additional
            # level factor (k):
            # FreqFeature_Normal[(j * Level + k) * NoOfFeature + feature_index, i]

            index_base = (j * Level + k) * NoOfFeature
            # Normal features
            FreqFeature_Normal[index_base + 0, i] = np.max(coef1[:, j])
            FreqFeature_Normal[index_base + 1, i] = np.min(coef1[:, j])
            FreqFeature_Normal[index_base + 2, i] = np.mean(coef1[:, j])
            FreqFeature_Normal[index_base + 3, i] = rms(coef1[:, j])
            FreqFeature_Normal[index_base + 4, i] = np.var(coef1[:, j])
            FreqFeature_Normal[index_base + 5, i] = sp.skew(coef1[:, j])
            FreqFeature_Normal[index_base + 6, i] = sp.kurtosis(coef1[:, j])
            FreqFeature_Normal[index_base + 7, i] = np.max(coef1[:, j]) / rms(
                coef1[:, j]
            )
            FreqFeature_Normal[index_base + 8, i] = rms(coef1[:, j]) / np.mean(
                np.abs(coef1[:, j])
            )
            FreqFeature_Normal[index_base + 9, i] = np.max(
                coef1[:, j]
            ) / np.mean(np.abs(coef1[:, j]))

            # Abnormal features
            FreqFeature_Abnormal[index_base + 0, i] = np.max(coef2[:, j])
            FreqFeature_Abnormal[index_base + 1, i] = np.min(coef2[:, j])
            FreqFeature_Abnormal[index_base + 2, i] = np.mean(coef2[:, j])
            FreqFeature_Abnormal[index_base + 3, i] = rms(coef2[:, j])
            FreqFeature_Abnormal[index_base + 4, i] = np.var(coef2[:, j])

```

```

FreqFeature_Abnormal[index_base + 5, i] = sp.skew(coef2[:, j])
FreqFeature_Abnormal[index_base + 6, i] = sp.kurtosis(coef2[:, j])
FreqFeature_Abnormal[index_base + 7, i] = np.max(
    coef2[:, j]
) / rms(coef2[:, j])
FreqFeature_Abnormal[index_base + 8, i] = rms(
    coef2[:, j]
) / np.mean(np.abs(coef2[:, j]))
FreqFeature_Abnormal[index_base + 9, i] = np.max(
    coef2[:, j]
) / np.mean(np.abs(coef2[:, j]))

#####
#####

print(FreqFeature_Normal.shape)
print(FreqFeature_Abnormal.shape)

FreqFeature_Normal
FreqFeature_Abnormal

```

(240, 180)

(240, 180)

```

Out[11]: array([[ 4.95512148e-01,  3.88689527e-01,  3.94501944e-01, ...,
                  1.74513954e-01,  2.60187011e-01,  3.96092935e-01],
                 [-3.95032274e-01, -6.90914036e-01, -3.34744350e-01, ...,
                  -1.66848916e-01, -3.19675905e-01, -4.35224224e-01],
                 [-1.45465834e-05,  1.96574156e-06, -2.23906458e-06, ...,
                  1.79478027e-06,  1.17358701e-05,  1.36559296e-06],
                 ...,
                 [ 1.40349966e+00,  1.37557594e+00,  1.37931123e+00, ...,
                  1.31653745e+00,  1.37566422e+00,  1.38869913e+00],
                 [ 1.13667955e+00,  1.12758813e+00,  1.13228230e+00, ...,
                  1.12796073e+00,  1.12732527e+00,  1.13028943e+00],
                 [ 1.59532937e+00,  1.55108310e+00,  1.56176969e+00, ...,
                  1.48500255e+00,  1.55082103e+00,  1.56963194e+00]],
          shape=(240, 180))

```

Combine Normal and Abnormal feature arrays

- axis=0: combine rows
- axis=1: combine columns

```

In [12]: FreqFeature = np.concatenate(
          [FreqFeature_Normal, FreqFeature_Abnormal], axis=1
        )
FreqFeature.shape

```

Out[12]: (240, 360)

Final Feature Dataset

- (30 Time domain features + 240 Frequency domain features = 270 features)

```

In [13]: Features = np.concatenate([TimeFeature, FreqFeature], axis=0)

print(Features.shape)
Features

```

(270, 360)

```
Out[13]: array([[ 1.35100000e+00,  3.16610000e+01,  3.18320000e+01, ...,
                  8.45460000e-01,  8.48450000e-01,  7.58330000e-01],
                [-1.37200000e+00, -2.27860000e+01, -2.36130000e+01, ...,
                  -6.56270000e-01, -7.47890000e-01, -9.14290000e-01],
                [ 1.10828100e-02,  2.33385723e-02,  2.05055037e-02, ...,
                  3.59270139e-02,  3.58358321e-02,  3.62789468e-02],
                ...,
                [ 1.37256701e+00,  1.39066341e+00,  1.36145813e+00, ...,
                  1.31653745e+00,  1.37566422e+00,  1.38869913e+00],
                [ 1.13646441e+00,  1.12338916e+00,  1.12893576e+00, ...,
                  1.12796073e+00,  1.12732527e+00,  1.13028943e+00],
                [ 1.55987356e+00,  1.56225620e+00,  1.53699878e+00, ...,
                  1.48500255e+00,  1.55082103e+00,  1.56963194e+00]],
              shape=(270, 360))
```

Convert Array into Data frame format

- Easy to save as data file (csv)

```
In [14]: Features_df = pd.DataFrame(Features)
         Features_df
```

```
Out[14]:
```

	0	1	2	3	4	5	6	7	8	9	...
0	1.351000	31.661000	31.832000	1.418300	1.053400	30.628000	0.992040	0.992420	1.059700	1.170800	...
1	-1.372000	-22.786000	-23.613000	-1.085600	-1.057500	-19.468000	-1.319600	-1.056400	-2.041700	-1.343200	...
2	0.011083	0.023339	0.020506	0.027215	0.016574	0.018563	0.020904	0.024480	0.029605	0.028426	...
3	0.426105	2.312749	2.313820	0.396240	0.388252	2.088591	0.403801	0.404898	0.381526	0.412919	...
4	0.181443	5.348262	5.353342	0.156266	0.150465	4.361866	0.162618	0.163343	0.144686	0.169694	...
...	...	...	...	...	...	...	...	...	...	...	...
265	-0.069441	-0.095775	-0.094919	-0.098230	-0.091625	-0.100346	-0.101488	-0.105314	-0.111518	-0.094917	...
266	-1.500603	-1.513689	-1.520788	-1.502072	-1.514240	-1.522188	-1.515150	-1.520913	-1.511260	-1.516616	...
267	1.372567	1.390663	1.361458	1.367672	1.364317	1.356064	1.355348	1.343550	1.382847	1.354853	...
268	1.136464	1.123389	1.128936	1.125794	1.126631	1.126265	1.125331	1.126091	1.121208	1.126622	...
269	1.559874	1.562256	1.536999	1.539718	1.537082	1.527288	1.525215	1.512959	1.550459	1.526407	...

270 rows × 360 columns



Save Final Feature Data in .csv

```
In [15]: path = "./data/FeatureData.csv"
         Features_df.to_csv(path, sep=";", header=None, index=None)
```